# Python Interview Questions

## Core Python syntax and fundamentals

1. What is Python, and what are its key features?
2. What is the difference between a compiler and an interpreter? How does Python execute code?
3. What are variables in Python? Do you need to declare their data types explicitly?
4. What are Python's built-in data types? Explain `int`, `float`, `str`, `bool`, `list`, `tuple`, `set`, and `dict`.
5. What is the difference between mutable and immutable data types? Give examples.
6. What is the difference between `==` and `is` in Python?
7. What is the difference between a list and a tuple?
8. What is type casting? Explain implicit and explicit type conversion.
9. What is the difference between `break`, `continue`, and `pass`?
10. What is the difference between `for` and `while` loops? When would you use each?

## Data structures, functions and practical programming

1. What is the difference between a list, tuple, set, and dictionary?
2. What is list slicing? Explain the syntax `list[start:stop:step]`.
3. What are list comprehensions? How are they different from ordinary `for` loops?
4. What is the difference between `append()`, `extend()`, and `insert()`?
5. What is the difference between `remove()`, `pop()`, and `del`?
6. What are `*args` and `**kwargs`? When would you use them?
7. What is the difference between a parameter and an argument?
8. What is the difference between `return` and `print()`?
9. What is a lambda function? Explain with an example.
10. What is the difference between local, global, and nonlocal variables?

SeionKAI

**Object-oriented programming, modules and error handling**

1. What is object-oriented programming (OOP)? Explain its four main principles.
2. What is the difference between a class and an object?
3. What is the purpose of the `__init__()` method?
4. What is the difference between instance methods, class methods, and static methods?
5. What is inheritance? Explain single, multiple, multilevel, and hierarchical inheritance.
6. What is method overriding? How does it differ from method overloading in Python?
7. What is the difference between public, protected, and private attributes in Python?
8. What is exception handling? Explain `try`, `except`, `else`, and `finally`.
9. What is the difference between a module, a package, and a library?
10. How do you create and raise a custom exception in Python?

**Python internals, functional programming and efficient code**

1. What is the difference between an iterable, an iterator, and a generator?
2. What is the `yield` keyword, and how does it differ from `return`?
3. What are decorators in Python? Explain how a function can modify another function's behaviour.
4. What are closures? How does a nested function retain access to variables from its enclosing scope?
5. What is the Global Interpreter Lock (GIL), and how does it affect multithreaded Python programs?
6. What is the difference between shallow copying and deep copying?
7. What is the difference between multithreading, multiprocessing, and asynchronous programming?
8. What are context managers? Explain the purpose of the `with` statement.
9. What are `__str__()` and `__repr__()`? How do they differ?

SelonixAI

10. What are Python's `map()`, `filter()`, and `reduce()` functions, and when are they useful?

SelonixAi